

Tarea 2.6. - Programación funcional II

EJERCICIO 1 – “Principal.java”, “Registro.java”, “IDescarga.java”, “IConvertir.java”

-Programa en Java el cruce de un fichero de alumnos y de un fichero de asignaturas que se llaman respectivamente entrada1.txt y entrada2.txt y que se encuentran alojados en un espacio de la universidad de valencia en las siguientes URLs:

- <https://mural.uv.es/franpevi/entrada1.txt>
- <https://mural.uv.es/franpevi/entrada2.txt>

-El formato de **entrada1.txt** consistirá en una línea con el número de registros que contiene el fichero seguido de una cabecera “NIA;NOMBRE” y de los datos relacionados con el NIA y el NOMBRE de varios alumnos:

```
7
NIA;NOMBRE
123;FRANCISCO
002;HUGO
456;RAUL
122;JOAQUIN
663;DIEGO
532;ALFONSO
001;DAVID
```

-Por su parte, el formato de **entrada2.txt** será de una línea con el número de registros que contiene el fichero seguido de una cabecera “NIA;ASIGNATURA” y de la relación entre el NIA de un alumno y el nombre de la asignatura en la que está matriculado.

```
13
NIA;ASIGNATURA
456;MATEMATICAS
456;INGLES
002;LENGUA
532;VALENCIANO
001;GEOGRAFIA
663;VALENCIANO
456;HISTORIA
123;HISTORIA
122;EDUCACION FISICA
122;MATEMATICAS
001;INGLES
663;FRANCES
532;FISICA
```

-El programa se encargará de leer estos 2 ficheros que están desordenados, ordenarlos por NIA y “enlazar” ambos ficheros, produciendo una salida por pantalla en la que se pueda ver en cada registro el NIA + NOMBRE + ASIGNATURA. Habrá que tener en cuenta que puede existir más de 1 asignatura por alumno.

El procedimiento para realizar el cruce tendrá que seguir los siguientes pasos:

PASO 1: DESCARGA DE FICHEROS

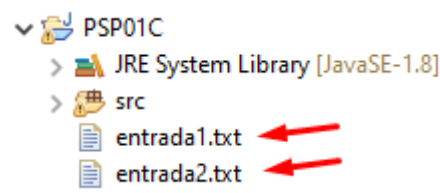
- Diseña un interfaz funcional (IDescarga) con solo 1 método que reciba como argumento de entrada una cadena de texto y que no devuelva nada. Codifica en tu clase Principal el cuerpo de ese método mediante una expresión lambda que tendrá los siguientes pasos:

- Crea un proceso (con ProcessBuilder) que se encargue de lanzar el bash y ejecutar el comando `wget` para recuperar el fichero que se le facilita en la entrada. Este comando `wget` recuperará el archivo de la dirección de `mural.uv.es` que se ha especificado antes.

- Dentro de esa misma expresión lambda y una vez hayamos lanzado el `wget`, nos esperaremos a que termine con el comando `waitFor`, que nos devolverá un 0 en caso de que el fichero se haya descargado con éxito.

- Ejecuta esa expresión lambda 2 veces, facilitándole primero en la entrada la cadena “entrada1.txt” para que descargue ese fichero y después facilitándole “entrada2.txt” para que también lo descargue.

- En el momento que se descarguen los ficheros, deberían aparecer en el explorador de Eclipse, justo debajo de la carpeta `src`. Si no aparecieran haz clic derecho sobre la carpeta del proyecto y selecciona la opción “Refresh” para actualizar el contenido de la carpeta.



PASO 2: CONVERSIÓN DE FICHEROS A ARRAYLIST

- Diseña una clase (Registro) que corresponderá a los registros que vamos leyendo de los ficheros de texto. Esta clase tendrá un atributo entero que será el “NIA” del alumno y luego un atributo tipo cadena de texto al que llamaremos “dato”, ambos serán los argumentos de entrada del constructor.

En cuanto al campo “dato”, le daremos este nombre tan genérico porque ese dato cuando se trata del fichero “entrada1” corresponde al nombre del alumno y cuando se trata de “entrada2” corresponderá al nombre de la asignatura. Codificamos además los correspondientes getters y setters de los atributos.

-Diseñamos un interfaz funcional(`IConvertir`) que se encarga de recibir como argumento de entrada un objeto de tipo `File` y que devolverá un `ArrayList` de objetos de tipo `Registro`. Codifica en tu clase `Principal` el cuerpo de ese método mediante una expresión lambda que seguirá los siguientes pasos:

- Lee del archivo la línea, cada registro leído se “splitea” con el carácter “;” y se crea un objeto con el NIA y el dato que se ha leído. Una vez creado el objeto, se añade al `ArrayList`.
- Una vez definida la expresión lambda, ejecútala 2 veces:
 - La primera vez, le facilitaremos en la entrada el objeto `File` “`entrada1.txt`” y el resultado lo almacenaremos en un `ArrayList` de objetos de tipo `Registro` al que llamaremos “`lista_alumnos`”.
 - La segunda vez, le facilitaremos en la entrada el objeto `File` “`entrada2.txt`” y el resultado lo almacenaremos en un `ArrayList` de objetos de tipo `Registro` al que llamaremos “`lista_asignaturas`”.
- De esta forma, ya tenemos en estructuras estáticas los datos de los ficheros que hemos descargado y podemos proceder al siguiente paso, que es la ordenación de esas estructuras.

PASO 3: ORDENACIÓN DE LOS <code>ARRAYLIST</code> DE ALUMNOS Y ASIGNATURAS POR EL NIA

-Vamos a ordenar los `ArrayList` de alumnos y asignaturas mediante expresiones lambda que corresponderán a interfaces `Runnable`s, por lo que las ordenaremos mediante hilos de ejecución.

-Para ello, codificaremos la primera expresión lambda, que hará referencia a la estructura “`listaAlumnos`” la cual se ordenará por el NIA y que dejará la lista ordenada en la estructura “`listaAlumnosSort`” que previamente habrá sido declarada en el programa `Principal`. Una vez definida, la metemos en un `Thread`, llamamos al método “`start`” y esperamos a que finalice el hilo (con “`join`”). Una vez ha finalizado, escribimos por pantalla los elementos almacenados en “`listaAlumnosSort`”.

-Procederemos del mismo modo para la estructura “`listaAsignaturas`”. Cuando finalicemos, la escribiremos también por pantalla, para que se vea el listado ordenado de asignaturas.

PASO 4: CRUCE ENTRE <code>ARRAYLIST</code> DE ALUMNOS Y DE ASIGNATURAS

- Una vez ordenados los `ArrayList`, nos encargaremos de realizar un cruce por el NIA, de forma que crearemos un `ArrayList` de tipo `Registro` donde almacenaremos el NIA que coincida en ambos vectores y en el campo dato introduciremos el nombre del alumno concatenado con el nombre de la asignatura en la que está matriculado.

- Una vez tenemos en el ArrayList los objetos de tipo "Registro" que se corresponden a los alumnos que se han matriculado en una o varias asignaturas, recorreremos la estructura para mostrar la información por pantalla mediante printf y, de esa forma, finalizará el programa.

Cuando se ejecute el programa "Principal" deberíamos ver la siguiente información:

- Confirmación de que el fichero "entrada1.txt" se ha descargado con éxito.
- Confirmación de que el fichero "entrada2.txt" se ha descargado con éxito.
- Impresión de la lista ordenada de alumnos.
- Impresión de la lista ordenada de asignaturas.
- Impresión del listado que contendrá el resultado de hacer el cruce entre las listas anteriores.

Un ejemplo de ejecución sería el siguiente, en el que los ficheros los hemos descargado con éxito e imprimimos los listados del siguiente modo:

Lista ordenada de alumnos:

```
1 : DAVID
2 : HUGO
122 : JOAQUIN
123 : FRANCISCO
456 : RAUL
532 : ALFONSO
663 : DIEGO
```

Lista ordenada de asignaturas:

```
1 : GEOGRAFIA
1 : INGLES
2 : LENGUA
122 : EDUCACION FISICA
122 : MATEMATICAS
123 : HISTORIA
456 : MATEMATICAS
456 : INGLES
456 : HISTORIA
532 : VALENCIANO
532 : FISICA
663 : VALENCIANO
663 : FRANCES
```

Lista resultado:

```
1 : DAVID GEOGRAFIA
1 : DAVID INGLES
2 : HUGO LENGUA
122 : JOAQUIN EDUCACION FISICA
122 : JOAQUIN MATEMATICAS
123 : FRANCISCO HISTORIA
456 : RAUL MATEMATICAS
456 : RAUL INGLES
456 : RAUL HISTORIA
532 : ALFONSO VALENCIANO
532 : ALFONSO FISICA
663 : DIEGO VALENCIANO
663 : DIEGO FRANCES
```

EJERCICIO 2 – “Principal.java”

-Realiza en Java un programa para la última película de Misión Imposible (cuyo subtítulo es “Operación Pentágono”) que le permitirá a Tom Cruise sabotear los servidores de la sede de defensa del gobierno de Estados Unidos con sólo unas líneas de código malicioso.

-Para ello, el programa solicitará al usuario el número de virus a inyectar en el Pentágono. Por cada uno de ellos, el programa funcionará del siguiente modo:

- Lanzará un hilo de ejecución (codificado en forma de expresión lambda) informando el número de virus (Virus 1, Virus 2, etc...)

- En la misma línea, se mostrará una barra de progreso en la que, en cada segundo, acumularemos de forma aleatoria un 5 o un 10, para de esa forma dotar de más realismo, si cabe, a la película.

- Cuando sumemos un 5 al total, imprimiremos una “X” en la barra de progreso y cuando sumemos 10, imprimiremos “XX”, esperando medio segundo entre impresión e impresión para ir viendo en tiempo real como se va completando.

- Cuando lleguemos al 100%, imprimiremos el valor “100%” al lado de la barra de progreso y pasaremos a darle el control al siguiente virus, que se comportará exactamente igual.

-Cuando se hayan cargado todos los virus, el programa imprimirá el mensaje “EL PENTÁGONO HA SIDO INFECTADO”.

Un ejemplo de ejecución sería el siguiente: si queremos cargar 4 virus en el Pentágono, primero se carga el virus 1, luego el virus 2...

```
Principal (177) [Java Application] C:\Program Files\JAVA'
```

```
MISSION IMPOSSIBLE: OPERACION PENTÁGONO
```

```
Introduzca numero de virus a cargar...
```

```
4
```

```
|
```

```
Virus 1:XXXXXXXXXXXXXXXXXXXXX 100%
```

```
Virus 2:XXXXXX
```

Y cuando se cargan todos los virus, finaliza el programa:

```
<terminated> Principal (177) [Java Application] C:\Program Files\JAVA'
```

```
MISSION IMPOSSIBLE: OPERACION PENTÁGONO
```

```
Introduzca numero de virus a cargar...
```

```
4
```

```
Virus 1:XXXXXXXXXXXXXXXXXXXXX 100%
```

```
Virus 2:XXXXXXXXXXXXXXXXXXXXX 100%
```

```
Virus 3:XXXXXXXXXXXXXXXXXXXXX 100%
```

```
Virus 4:XXXXXXXXXXXXXXXXXXXXX 100%
```



